WebSockets made easy

with OpenSwoole

by **Savio Resende**

PHPTek 2026

Talk Objective

1. **What** are WebSockets and how they work?
2. **How** to use WebSockets? (OpenSwoole context)
3. **Why** should you use WebSockets? (with OpenSwoole)

HTTP

Request → Response. One shot. Client drives.

WebSocket Handshake

> WebSocket connections are HTTP connections upgraded.

1. `wss://` request sent with `Upgrade: websocket`
2. Server responds **101 Switching Protocols**
3. Persistent, bidirectional channel open — no more HTTP overhead

The Upgrade Request

```
GET / HTTP/1.1
Host: savioresende.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: {...}
Sec-WebSocket-Version: 13
Sec-WebSocket-Protocol: socketconveyor.com
```

Advantages vs Trade-Offs

Advantages

- **Bidirectional** — server pushes without prompt
- **Efficiency** — faster than HTTP alternatives
- **Low latency** — no handshake per message
- **Low overhead** — small frame headers

Trade-Offs

- **Complexity** — stateful connections, reconnection, backpressure
- **Security** — new surface area, new rules

Scale & Security

Message Broker

Origin Verification

SSL Encryption

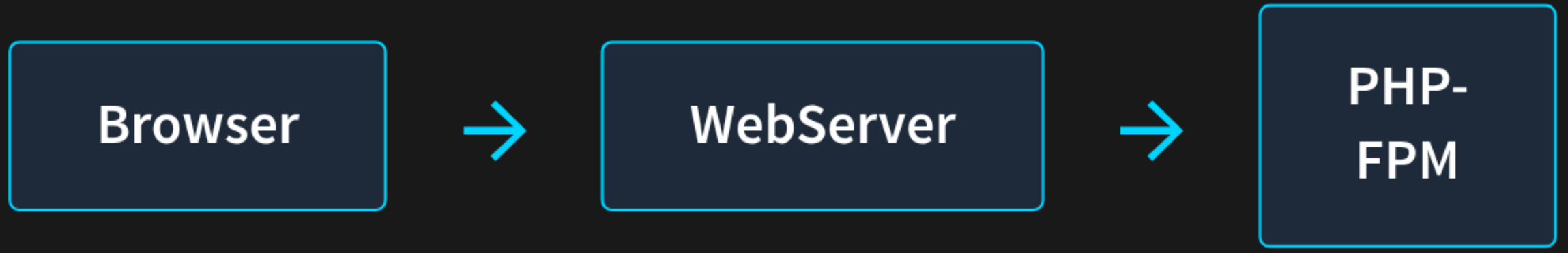
What is OpenSwoole?

- Asynchronous
- Parallel
- High-performance
- Coroutine-based
- Communication engine

TCP / HTTP / WebSocket servers — programmatically.

How Is That Different?

Traditional



With OpenSwoole

Reference: openswoole.com/how-it-works

Alternatives Comparison

Ping/Pong — 1,000 clients, 2,000 messages each. Tool: Locust.

OpenSwoole

0.42

ms avg

Ratchet

1.93

ms avg

459% faster

OpenSwoole HTTP Server

```php
use OpenSwoole\HTTP\Server;

$server = new Server("0.0.0.0", 9501);

$server->on("request", function ($request, $response) {
    $response->header("Content-Type", "text/plain");
    $response->end("Hello from OpenSwoole\n");
});

$server->start();
```

OpenSwoole WebSocket Server

```php
use OpenSwoole\WebSocket\Server;

$server = new Server("0.0.0.0", 9502);

$server->on("open", function ($server, $request) {
    echo "Connection {$request->fd} opened\n";
});

$server->on("message", function ($server, $frame) {
    $server->push($frame->fd, "echo: {$frame->data}");
});

$server->on("close", function ($server, $fd) {
    echo "Connection {$fd} closed\n";
});
```

Async Scheduling

```php
use OpenSwoole\Timer;
use OpenSwoole\Coroutine;

// every 5 seconds
Timer::tick(5000, fn () => doWork());

// once after 5 seconds
Timer::after(5000, fn () => doWorkOnce());
```

Live Demos

Open in a new tab — server running on port 8989

WebSocket Echo

send a message → server echoes back in real-time

Channel Broadcast

open two tabs → messages appear on both instantly

Async Timer Push

server pushes a timestamp every 2 seconds

Conveyor Control Panel

button-driven client for the Conveyor

Resources

- kanata-php/socket-conveyor
- MDN WebSockets Documentation (https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API)

Thank You

savioresende.com

@lotharthesavior